American University Of The Middle East

Project Report

Project Deliverable 1 (PD1) – Technical Report

Signals and Systems EE/CE 301

Summer 2026

Project Title: Signal & System Analysis and the Effect of Noise

Section: M1 Group: 5

Name	ID	Major
Yousef AlAbdulrazzaq	75018	CE
Jamal AlEleawah	10094770	EE
Abdelrahman Hussein	10095516	EE

Instructor: Dr. Yazid Khattabi

Date: **24/06/2026**

1. Introduction

1.1 Problem Definition

A signal $x_1(t) = \cos(0.5\pi t)$ V is received from a source. In its received form the signal is not suitable for the receiver, so it must first be transformed into the desired format $x_2(t) = \sin(1.5\pi t)$ V using the signal-transformation operations studied in the course (time shifting, scaling, and reversal). The transformed signal $x_2(t)$ is then applied as the input to a predefined system that performs a time delay of 2 s, $y(t) = x(t - 2)$, whose causality is to be investigated. Finally, $x_2(t)$ is transmitted through a medium that adds a DC noise signal $n(t) = 50$ mV, producing the received signal $y(t) = x_2(t) + n(t)$; this noisy signal is analyzed in both the time and frequency domains, and a filter is designed to recover the transmitted signal. All parameters used in this report correspond to Section M1, Group 5 of Table 1 in the project brief.

1.2 Objectives

- Apply time-scaling and time-shifting operations to convert $x_1(t)$ into $x_2(t)$, and verify the result using MATLAB.
- Investigate the causality of the given time-delay system $y(t) = x(t - 2)$.
- Study the effect of additive DC noise on $x_2(t)$ in the time and frequency domains using the Fourier series.
- Design a filter that recovers the transmitted signal and determine its type, element values, and cutoff frequency.

1.3 Structure of the Report

Section 2 presents the contents and findings for the three tasks: signal transformation, system-property testing, and the study of noise together with the recovery-filter design. Each task is supported by theoretical calculations, hand sketches, and MATLAB simulations. Section 3 summarizes the work and outlines possible future extensions, Section 4 lists the references used, and Appendix A provides the complete MATLAB code.

2. Contents and Findings

2.1 Task 1 – Signal Transformation

For Section M1, Group 5 the received and desired signals are $x_1(t) = \cos(0.5\pi t)$ V and $x_2(t) = \sin(1.5\pi t)$ V. The aim is to express $x_2(t)$ as a combination of transformations applied to $x_1(t)$. Starting from the desired signal and converting the sine into a cosine:

$$x_2(t) = \sin(1.5\pi t) = \cos(1.5\pi t - \pi/2) = \cos(0.5\pi(3t - 1)) = x_1(3t - 1)$$

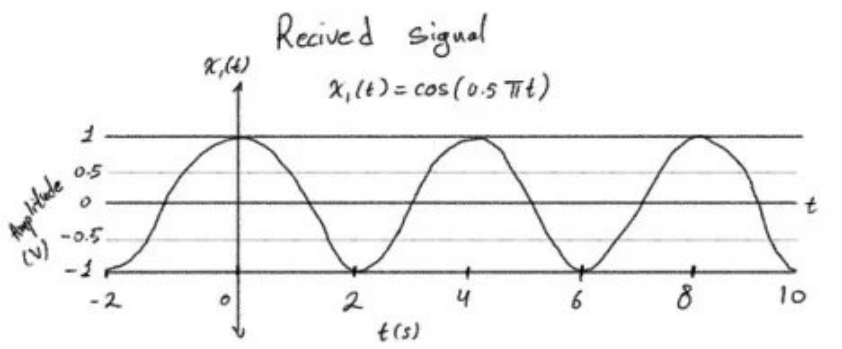
Therefore, $x_2(t) = x_1(3t - 1) = x_1(3(t - 1/3))$. This can be realised in two ordered steps:

- Step 1 – Time scaling (compression by a factor of 3): $x_1(t) \rightarrow x_1(3t) = \cos(1.5\pi t)$.
- Step 2 – Time shifting (delay by 1/3 s): $x_1(3t) \rightarrow x_1(3(t - 1/3)) = \sin(1.5\pi t)$.

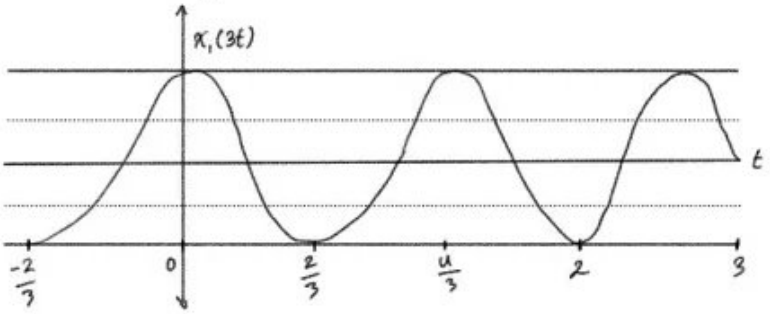
An equivalent ordering is to shift first and then scale: delaying $x_1(t)$ by 1 s gives $\cos(0.5\pi(t-1)) = \sin(0.5\pi t)$, and compressing the result by 3 again yields $\sin(1.5\pi t)$. No time reversal (flip) is required because the scaling factor (3) is positive.

Verification: $\cos(1.5\pi t - \pi/2) = \cos(1.5\pi t)\cos(\pi/2) + \sin(1.5\pi t)\sin(\pi/2) = \sin(1.5\pi t)$, which confirms the transformation.

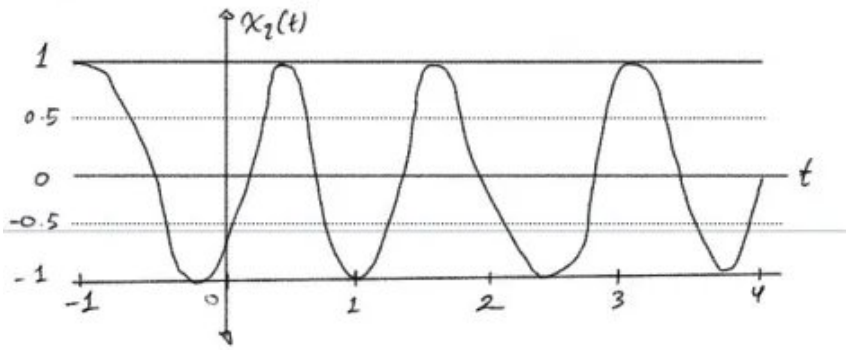
The periods of the two signals are $T_1 = 2\pi/(0.5\pi) = 4$ s and $T_2 = 2\pi/(1.5\pi) = 4/3$ s. As required, between three and ten periods should be drawn; for example, $t \in [0, 4]$ s contains three full periods of $x_2(t)$.



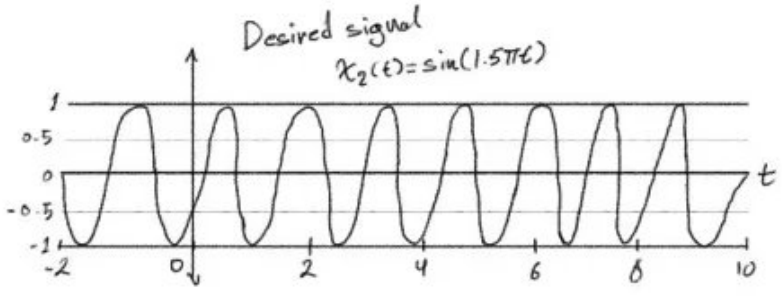
Step 1: Time scaling $x_1(t) \rightarrow x_1(3t) = \cos(1.5\pi t)$



Step 2: Time shift by $\frac{1}{3}$ seconds (Phase shift of $\frac{\pi}{2}$)



$x_2(t) = x_1(3t - 1) \leftarrow x_1(3(t - \frac{1}{3}))$.



```
%  TASK 1: SIGNAL TRANSFORMATION  x1(t) -> x2(t)
% Extended the time vector to 10s so we can see at least 3 periods of the slower x1(t)
t1 = -2:0.001:10;
x1 = cos(0.5*pi*t1);
x2_target = sin(1.5*pi*t1);

% Applying the transformation x2(t) = x1(3t - 1) we derived in the theory part
x2_built = cos(0.5*pi*(3*t1 - 1));

figure('Name','Task 1: Signal Transformation');

subplot(3,1,1);
plot(t1, x1, 'b', 'LineWidth', 1.4); grid on;
title('Received signal x_1(t) = cos(0.5\pi t)');
xlabel('t (s)'); ylabel('Amplitude (V)');
legend('x_1(t)', 'Location', 'best');

subplot(3,1,2);
plot(t1, x2_target, 'r', 'LineWidth', 1.4); grid on;
title('Desired signal x_2(t) = sin(1.5\pi t)');
xlabel('t (s)'); ylabel('Amplitude (V)');
legend('x_2(t)', 'Location', 'best');

subplot(3,1,3);
plot(t1, x2_target, 'r', 'LineWidth', 2); hold on;
plot(t1, x2_built, 'k--', 'LineWidth', 1.2); grid on;
title('Verification: x_2(t) vs. x_1(3t-1)');
xlabel('t (s)'); ylabel('Amplitude (V)');
legend('x_2(t) target', 'x_1(3t-1) built', 'Location', 'best');

% Checking if the built signal matches the target perfectly
max_error_task1 = max(abs(x2_target - x2_built));
fprintf('Task 1: max error between x2(t) and x1(3t-1) = %.4g\n', max_error_task1);
```

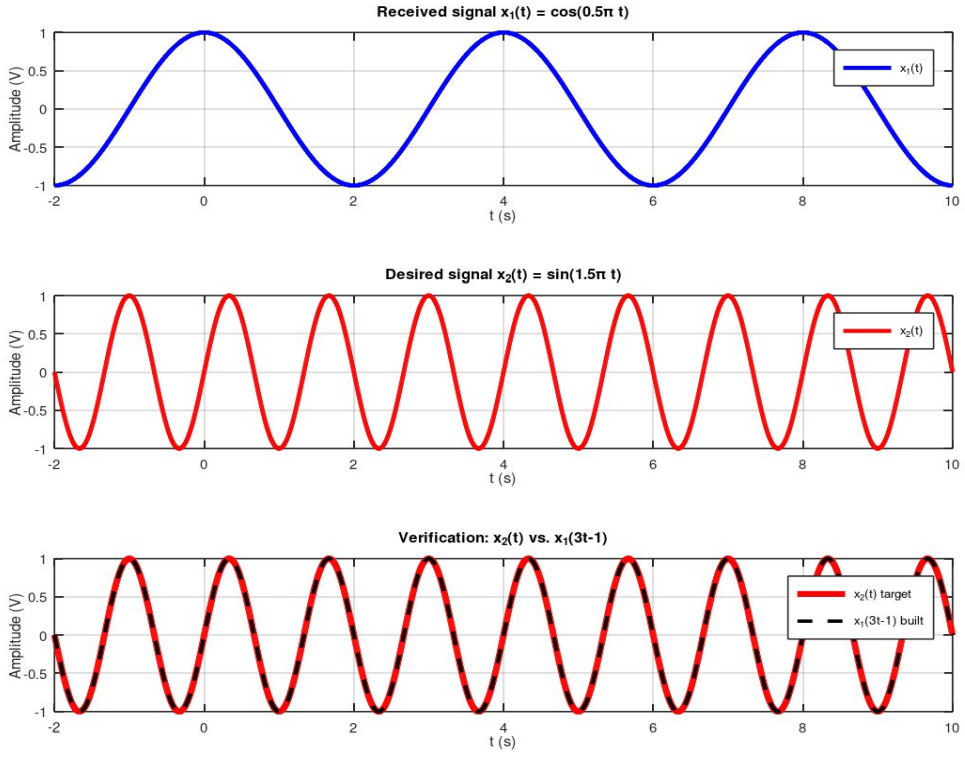


Figure 1. Signal transformation $x_2(t) = x_1(3t - 1)$: MATLAB verification.

2.2 Task 2 – System Property: Causality

The system of interest performs a time delay of 2 s, described by y(t) = x(t − 2). The property to be investigated is causality.

Definition: a system is causal if its output at any time t depends only on the values of the input at the present and past instants (times ≤ t), and never on future inputs.

For this system the output y(t) is equal to the input evaluated at t − 2. Since t − 2 < t for every t, the output depends only on a past value of the input. The system is therefore causal. By contrast, an advance system y(t) = x(t + 2) would require a future input value (t + 2 > t) and would be non-causal.

Illustration: if the input is zero for t < 0, the output remains zero until t = 2 s; the system never responds before the input arrives, which is consistent with a causal system.

```
%  TASK 2: SYSTEM PROPERTY - CAUSALITY

t2 = -1:0.001:8;
% Multiplying by a step function so the input clearly starts at t=0
x_in  = sin(1.5*pi*t2) .* (t2 >= 0);
% Output is just the input delayed by 2s, also zero before t=2
y_out = sin(1.5*pi*(t2-2)) .* ((t2-2) >= 0);

figure('Name','Task 2: Causality Test');
plot(t2, x_in, 'b', 'LineWidth', 1.4); hold on;
plot(t2, y_out, 'r--', 'LineWidth', 1.4); grid on;
xlabel('t (s)'); ylabel('Amplitude (V)');
title('Causality test for y(t) = x(t-2)');
legend('Input x(t), starts at t = 0', 'Output y(t), starts at t = 2',
       'Location', 'best');
```

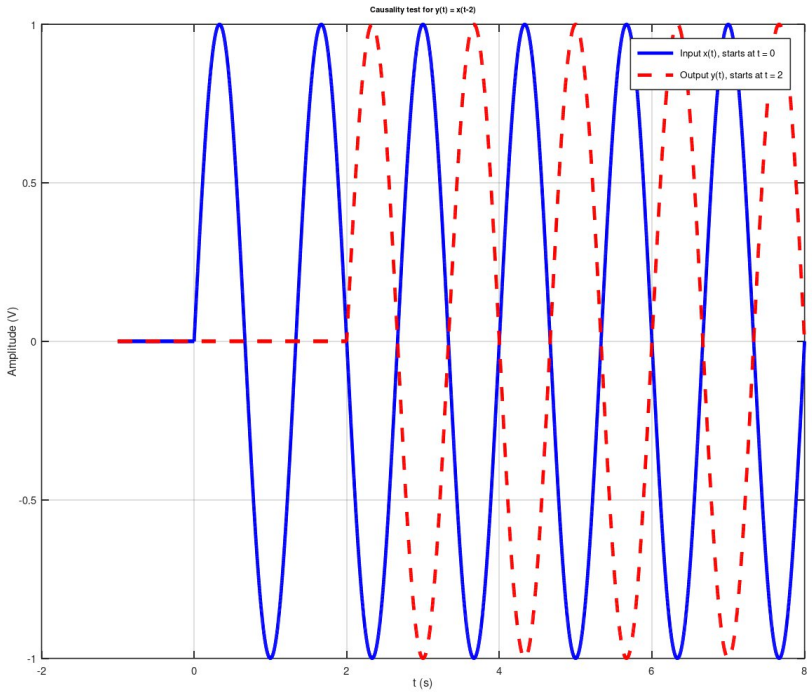


Figure 2. Causality test for y(t) = x(t − 2).

2.3 Task 3 – Effect of Noise

The transformed signal $x_2(t) = \sin(1.5\pi t)$ is transmitted through a medium that adds the noise n(t) = 50 mV = 0.05 V, which is a constant (DC) signal. The received signal is $y(t) = x_2(t) + n(t)$. The fundamental angular frequency is $\omega_0 = 1.5\pi$ rad/s, giving a period $T_0 = 4/3$ s and a fundamental frequency $f_0 = \omega_0/(2\pi) = 0.75$ Hz.

Using the exponential Fourier series $x(t) = \Sigma\, a_k\, e^{jk\omega_0 t}$ and the identity $\sin(\omega_0 t) = (e^{j\omega_0 t} - e^{-j\omega_0 t})/(2j)$:

- Signal $x_2(t)$: $a_1 = -j/2$ and $a_{-1} = +j/2$, with all other coefficients zero; hence $|a_{\pm 1}| = 0.5$.
- Noise n(t): being a DC value, only the average term is non-zero, $b_0 = 0.05$ V, with all other $b_k = 0$.
- Received signal y(t): $c_k = a_k + b_k$, so $c_0 = 0.05$, $c_1 = -j/2$ and $c_{-1} = +j/2$.

The magnitudes of the Fourier coefficients are summarized below (all coefficients for $|k| \geq 2$ are zero).

k	$\lvert a_k \rvert$ (signal)	$\lvert b_k \rvert$ (noise)	$\lvert c_k \rvert$ (received)
−1	0.5	0	0.5
0	0	0.05	0.05
+1	0.5	0	0.5

Table 1. Fourier-series coefficient magnitudes of $x_2(t)$, n(t), and y(t).

Magnitude spectra over $k \in [-6, 6]$: $|a_k|$ shows two lines at k = ±1 of height 0.5; $|b_k|$ shows a single line at k = 0 of height 0.05; and $|c_k|$ is the superposition of the two, i.e. lines at k = ±1 (0.5) plus a DC line at k = 0 (0.05).

```
%  TASK 3: EFFECT OF NOISE - FOURIER SERIES, SPECTRA, AND FILTER DESIGN

% Setting up fundamental freq and period for the Fourier series
w0 = 1.5*pi;
f0 = w0/(2*pi);
T0 = 1/f0;

% 3a. Fourier series coefficients and magnitude spectra (k = -6..6)
k = -6:6;
a_k = zeros(size(k));
b_k = zeros(size(k));

% Plugging in the theoretical values for the signal and the DC noise
a_k(k == 1)  = -1i/2;
a_k(k == -1) =  1i/2;
b_k(k == 0)  = 0.05;

% The received signal coefficients are just the sum of the signal and noise
c_k = a_k + b_k;

figure('Name','Task 3a: Magnitude Spectra');

subplot(3,1,1);
stem(k, abs(a_k), 'filled', 'b'); grid on;
xlabel('k'); ylabel('|a_k|');
title('Magnitude spectrum of x_2(t)');

subplot(3,1,2);
stem(k, abs(b_k), 'filled', 'r'); grid on;
xlabel('k'); ylabel('|b_k|');
title('Magnitude spectrum of noise n(t)');

subplot(3,1,3);
stem(k, abs(c_k), 'filled', 'k'); grid on;
xlabel('k'); ylabel('|c_k|');
title('Magnitude spectrum of received signal y(t) = x_2(t) + n(t)');
```

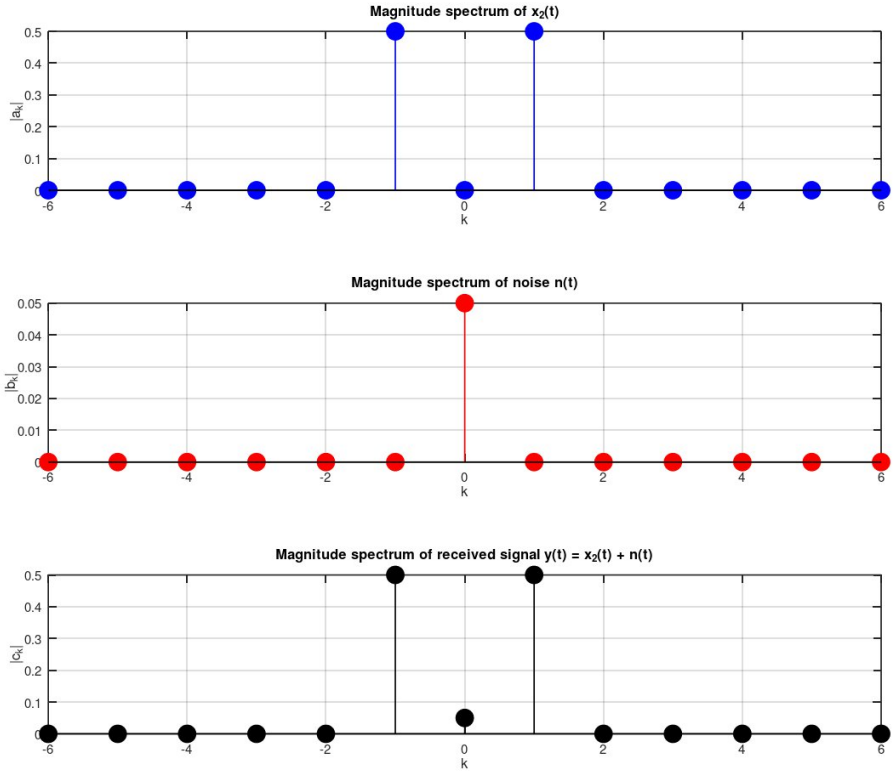


Figure 3. Magnitude spectra of the signal, noise, and received signal.

Effect of multiplying the noise by 10: the noise becomes n(t) = 0.5 V, so y(t) = sin(1.5πt) + 0.5. In the time domain the sinusoid keeps the same shape and amplitude, but the whole waveform is shifted upward, riding on a larger DC offset (the baseline rises from 0.05 V to 0.5 V). In the frequency domain only the k = 0 line grows (from 0.05 to 0.5); the k = ±1 lines are unchanged.

```
% 3b. Effect of multiplying the noise by 10

t3 = 0:0.001:2*T0;
x2_t = sin(w0*t3);

n_orig = 0.05*ones(size(t3));
n_x10  = 0.5*ones(size(t3));

y_orig = x2_t + n_orig;
y_x10  = x2_t + n_x10;

figure('Name','Task 3b: Noise x10 effect');
plot(t3, y_orig, 'b', 'LineWidth', 1.4); hold on;
plot(t3, y_x10,  'r', 'LineWidth', 1.4); grid on;
xlabel('t (s)'); ylabel('Amplitude (V)');
title('Effect of multiplying the noise by 10');
legend('y(t) = x_2(t) + 0.05 V', 'y(t) = x_2(t) + 0.5 V', 'Location', 'best
```

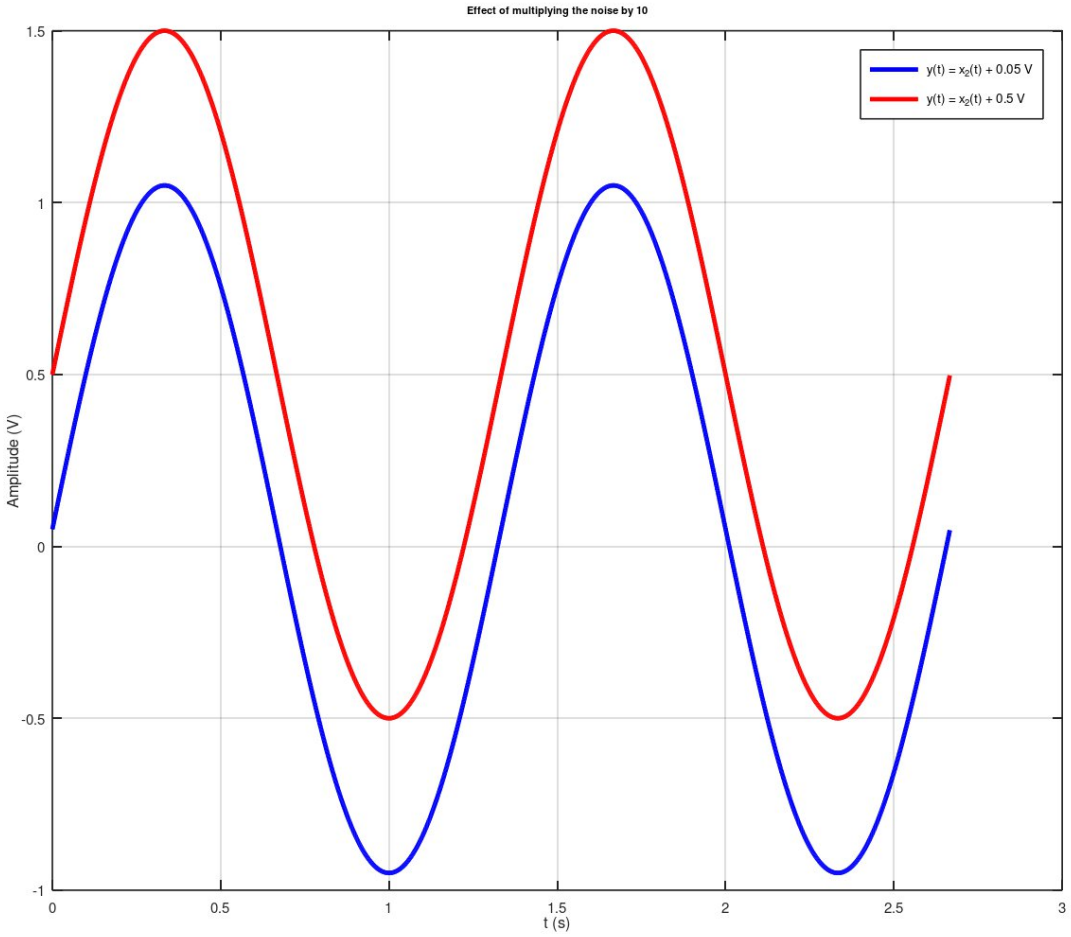
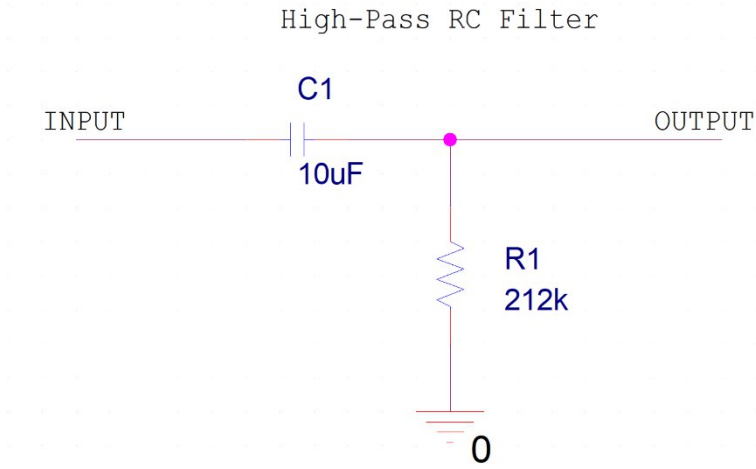


Figure 4. Time-domain effect of multiplying the noise by 10.

Filter design for signal recovery: the desired signal lies entirely at f_0 = 0.75 Hz, while the noise is located at DC (0 Hz). To recover $x_2(t)$ the DC component must be removed while 0.75 Hz is passed, so a first-order passive RC high-pass filter is proposed, with transfer function H(jω) = jωRC / (1 + jωRC). This filter has zero gain at DC and a gain approaching unity at high frequency. Choosing the cutoff a decade below the signal frequency, f_c = 0.075 Hz, and selecting C = 10 μF gives R = 1/(2π f_c C) ≈ 212 kΩ. At f_0 = 0.75 Hz the gain is approximately 0.995, so the signal passes almost unattenuated while the DC noise is completely blocked. The cutoff frequency of the filter is therefore f_c ≈ 0.075 Hz.




```
% 3c. Recovery filter design: first-order RC high-pass filter

% Picking fc a decade below the signal freq so it blocks DC but passes 0.75Hz
fc = f0/10;
wc = 2*pi*fc;

% Using a standard 10uF capacitor and calculating the required resistor value
C = 10e-6;
R = 1/(wc*C);

fprintf('Task 3c: Filter design -> fc = %.4f Hz, C = %.1f uF, R = %.1f kOhm\n',
        fc, C*1e6, R/1e3);

% Calculating the filter response at each harmonic k
H_k = (1i*k*w0*R*C) ./ (1 + 1i*k*w0*R*C);

figure('Name','Task 3c: Filter Frequency Response');
freq_axis = -1:0.001:1;
w_sweep = 2*pi*freq_axis;
H_sweep = (1i*w_sweep*R*C) ./ (1 + 1i*w_sweep*R*C);

plot(freq_axis, abs(H_sweep), 'LineWidth', 1.4); grid on; hold on;
plot(f0, abs(H_k(k==1)), 'ro', 'MarkerFaceColor', 'r');
xlabel('Frequency (Hz)'); ylabel('|H(f)|');
title('RC High-Pass Filter - Magnitude Response');
legend('|H(f)|', 'Signal frequency f_0 = 0.75 Hz', 'Location', 'best');
```

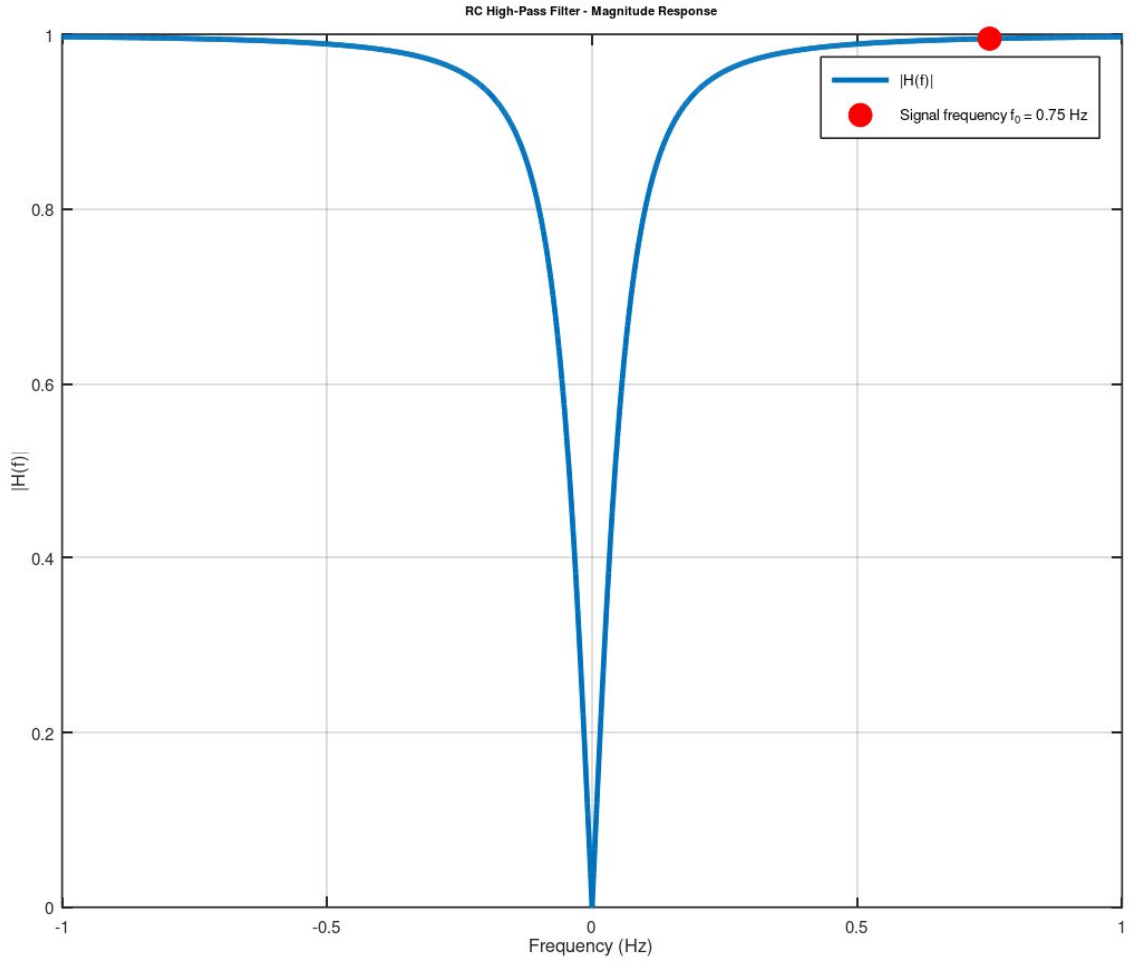


Figure 5. RC high-pass recovery filter and its response.

To verify filter performance, the Fourier coefficients c_k of the received signal y(t) were multiplied by the filter gain $H(jk\omega_0)$ at the corresponding harmonic, and the result was reconstructed in the time domain. Figure 6 compares the recovered signal to the original $x_2(t)$ and the noisy y(t).

```
% 3d. Recovered signal after filtering
t4 = 0:0.001:2*T0;
y_recovered = zeros(size(t4));

% Reconstructing the time-domain signal by multiplying coefficients with filter gain
for idx = 1:length(k)
    y_recovered = y_recovered + (c_k(idx)*H_k(idx)) * exp(1i*k(idx)*w0*t4);
end
% Dropping the imaginary part since it's just numerical noise
y_recovered = real(y_recovered);

figure('Name','Task 3d: Signal Recovery');
plot(t4, x2_t, 'b', 'LineWidth', 1.6); hold on;
plot(t4, y_orig, 'r:', 'LineWidth', 1.2);
plot(t4, y_recovered, 'k--', 'LineWidth', 1.6); grid on;
xlabel('t (s)'); ylabel('Amplitude (V)');
title('Signal recovery using the RC high-pass filter');
legend('Original x_2(t)', 'Noisy received y(t)', 'Filtered / recovered output',
       'Location', 'best');
```

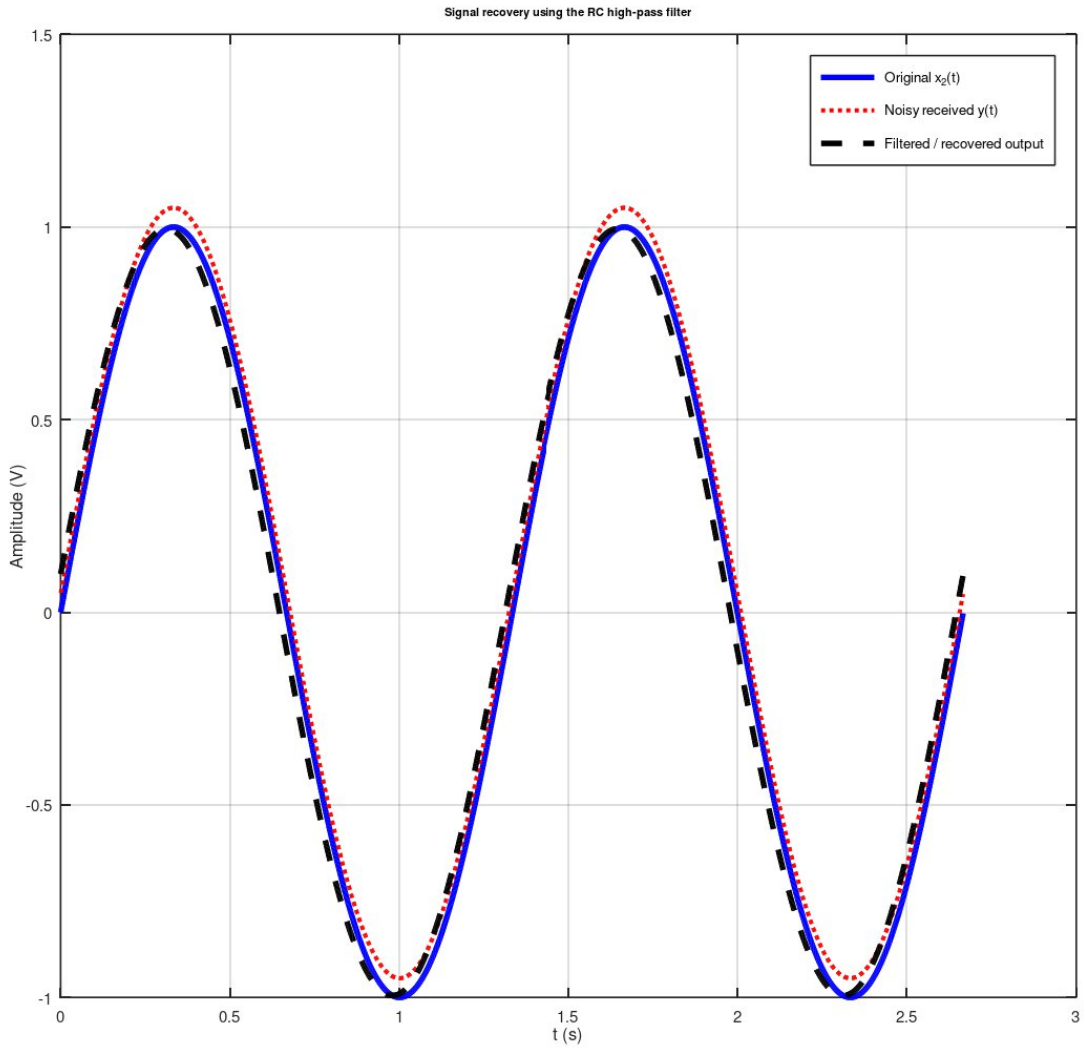


Figure 6. Signal recovery using the RC high-pass filter.

3. Conclusion

This report analyzed a signal-processing problem in three parts using the parameters of Section M1, Group 5. The received signal $x_1(t) = \cos(0.5\pi t)$ was transformed into $x_2(t) = \sin(1.5\pi t)$ through a time compression by a factor of 3 followed by a delay of 1/3 s, expressed compactly as $x_2(t) = x_1(3t - 1)$. The time-delay system $y(t) = x(t - 2)$ was shown to be causal because its output depends only on past inputs. Finally, the addition of a 50 mV DC noise was studied in the time and frequency domains; the Fourier analysis showed the noise as a DC ($k = 0$) component, and a first-order RC high-pass filter with $f_c \approx 0.075$ Hz ($C = 10$ μF, $R \approx 212$ kΩ) was designed to recover the transmitted signal. The MATLAB simulations are used throughout to verify the theoretical results.

Future Work

- Replace the idealized DC noise with random (e.g. Gaussian) noise and evaluate filter performance.
- Compare the first-order RC filter with higher-order or active filters for sharper roll-off.
- Extend the analysis to the audio signals studied in Part 2 of the project.

AI Use Disclaimer: AI tools were used to help draft, format, and check parts of this report. However, the group reviewed all content against course materials, and the final analysis, results, and conclusions are entirely the group's own.

4. References

Lathi, B. P., & Ding, Z. (2017). *Linear systems and signals* (3rd ed.). Oxford University Press.

The MathWorks. (n.d.). *MATLAB documentation*. https://www.mathworks.com/help/matlab/

Oppenheim, A. V., Willsky, A. S., & Nawab, S. H. (1997). *Signals and systems* (2nd ed.). Prentice Hall.

Appendix A: MATLAB Code

```
%% ======================================================================
%  PD1 - Technical Report: Theoretical Calculations & Simulation Results
%  Signals and Systems - EE/CE 301 - Summer 2026
%  Section: M1   Group: 5
%
%  Parameters (Table 1, Section M, Odd, row "1, 5"):
%   x1(t) = cos(0.5*pi*t)     [V]
%   x2(t) = sin(1.5*pi*t)     [V]
%   System: y(t) = x(t-2)     (time delay by 2 s)
%   Property to test: Causality
%   Noise: n(t) = 50 mV (constant/DC)
% ======================================================================

clear; clc; close all;

%% ======================================================================
%  TASK 1: SIGNAL TRANSFORMATION  x1(t) -> x2(t)
%  Theory: x2(t) = sin(1.5*pi*t) = x1(3t - 1) = x1(3*(t - 1/3))
%         i.e. compress x1 by a factor of 3, then delay by 1/3 s.
% ======================================================================

t1 = -2:0.001:6;                % time vector, covers >=3 periods of x1 & x2
x1 = cos(0.5*pi*t1);              % original received signal
x2_target = sin(1.5*pi*t1);         % desired signal (ground truth)

% Build x2 FROM x1 using the derived transformation x2(t) = x1(3t - 1)
x2_built = cos(0.5*pi*(3*t1 - 1));    % should equal x2_target if correct

figure('Name','Task 1: Signal Transformation');

subplot(3,1,1);
plot(t1, x1, 'b', 'LineWidth', 1.4); grid on;
title('Received signal x_1(t) = cos(0.5\pi t)');
xlabel('t (s)'); ylabel('Amplitude (V)');
legend('x_1(t)', 'Location', 'best');

subplot(3,1,2);
plot(t1, x2_target, 'r', 'LineWidth', 1.4); grid on;
title('Desired signal x_2(t) = sin(1.5\pi t)');
xlabel('t (s)'); ylabel('Amplitude (V)');
legend('x_2(t)', 'Location', 'best');

subplot(3,1,3);
plot(t1, x2_target, 'r', 'LineWidth', 2); hold on;
plot(t1, x2_built, 'k--', 'LineWidth', 1.2); grid on;
title('Verification: x_2(t) vs. x_1(3t-1)');
xlabel('t (s)'); ylabel('Amplitude (V)');
legend('x_2(t) target', 'x_1(3t-1) built', 'Location', 'best');

% Numerical check that the transformation is exact (up to floating point)
max_error_task1 = max(abs(x2_target - x2_built));
fprintf('Task 1: max error between x2(t) and x1(3t-1) = %.4g\n', max_error_task1);


%% ======================================================================
%  TASK 2: SYSTEM PROPERTY - CAUSALITY
%  System: y(t) = x(t-2)   (pure time delay of 2 s)
%  A causal system's output at time t depends only on the input at
```

```
%  times <= t (never on future values of the input).
% ========================================================================

t2 = -1:0.001:8;

% Test input: zero for t < 0, so we can see whether the output ever
% reacts BEFORE the input turns on (that would indicate non-causality).
x_in  = sin(1.5*pi*t2) .* (t2 >= 0);

% System output: y(t) = x(t-2)
y_out = sin(1.5*pi*(t2-2)) .* ((t2-2) >= 0);

figure('Name','Task 2: Causality Test');
plot(t2, x_in, 'b', 'LineWidth', 1.4); hold on;
plot(t2, y_out, 'r--', 'LineWidth', 1.4); grid on;
xlabel('t (s)'); ylabel('Amplitude (V)');
title('Causality test for y(t) = x(t-2)');
legend('Input x(t), starts at t = 0', 'Output y(t), starts at t = 2', ...
      'Location', 'best');

% The output stays at zero until t = 2 s (input start + 2 s delay).
% It never anticipates a future input value -> the system is CAUSAL.
disp('Task 2: Output remains zero until t = 2 s (input start + delay).');
disp('        The system never uses future input values -> CAUSAL.');


%% ========================================================================
%  TASK 3: EFFECT OF NOISE - FOURIER SERIES, SPECTRA, AND FILTER DESIGN
%  y(t) = x2(t) + n(t),   n(t) = 0.05 V (DC),   x2(t) = sin(1.5*pi*t)
% ========================================================================

w0 = 1.5*pi;            % fundamental angular frequency of x2(t) [rad/s]
f0 = w0/(2*pi);           % fundamental frequency [Hz]   (= 0.75 Hz)
T0 = 1/f0;                % fundamental period [s]       (= 4/3 s)

% ------------------------------------------------------------------------
% 3a. Fourier series coefficients and magnitude spectra (k = -6..6)
% ------------------------------------------------------------------------
k = -6:6;                    % harmonic index range required by the brief
a_k = zeros(size(k));           % Fourier coefficients of x2(t)
b_k = zeros(size(k));           % Fourier coefficients of n(t)

% sin(w0 t) = (e^{j w0 t} - e^{-j w0 t}) / (2j)  ->  a_1 = -j/2, a_-1 = +j/2
a_k(k == 1)  = -1i/2;
a_k(k == -1) =  1i/2;
b_k(k == 0)  = 0.05;            % constant (DC) noise -> only the k=0 term

c_k = a_k + b_k;                % Fourier coefficients of y(t) = x2(t) + n(t)

figure('Name','Task 3a: Magnitude Spectra');

subplot(3,1,1);
stem(k, abs(a_k), 'filled', 'b'); grid on;
xlabel('k'); ylabel('|a_k|');
title('Magnitude spectrum of x_2(t)');

subplot(3,1,2);
stem(k, abs(b_k), 'filled', 'r'); grid on;
xlabel('k'); ylabel('|b_k|');
title('Magnitude spectrum of noise n(t)');
```

```
subplot(3,1,3);
stem(k, abs(c_k), 'filled', 'k'); grid on;
xlabel('k'); ylabel('|c_k|');
title('Magnitude spectrum of received signal y(t) = x_2(t) + n(t)');

% ------------------------------------------------------------------------
% 3b. Effect of multiplying the noise by 10
% ------------------------------------------------------------------------
t3 = 0:0.001:2*T0;              % a couple of periods for clear viewing
x2_t = sin(w0*t3);

n_orig = 0.05*ones(size(t3));
n_x10  = 0.5*ones(size(t3));

y_orig = x2_t + n_orig;
y_x10  = x2_t + n_x10;

figure('Name','Task 3b: Noise x10 effect');
plot(t3, y_orig, 'b', 'LineWidth', 1.4); hold on;
plot(t3, y_x10,  'r', 'LineWidth', 1.4); grid on;
xlabel('t (s)'); ylabel('Amplitude (V)');
title('Effect of multiplying the noise by 10');
legend('y(t) = x_2(t) + 0.05 V', 'y(t) = x_2(t) + 0.5 V', 'Location', 'best');

% Comment: increasing the noise only raises the DC baseline of y(t);
% the AC (sinusoidal) part contributed by x2(t) is unaffected in shape.

% ------------------------------------------------------------------------
% 3c. Recovery filter design: first-order RC high-pass filter
%     H(jw) = jwRC / (1 + jwRC)
% ------------------------------------------------------------------------
fc = f0/10;               % cutoff a decade below the signal frequency
wc = 2*pi*fc;             % cutoff angular frequency [rad/s] (~0.075 Hz)

C = 10e-6;                % chosen capacitor value [F]
R = 1/(wc*C);             % resulting resistor value [Ohm]

fprintf('Task 3c: Filter design -> fc = %.4f Hz, C = %.1f uF, R = %.1f kOhm\n', ...
        fc, C*1e6, R/1e3);

% Filter response evaluated exactly at each harmonic frequency k*w0
H_k = (1i*k*w0*R*C) ./ (1 + 1i*k*w0*R*C);

figure('Name','Task 3c: Filter Frequency Response');
freq_axis = -1:0.001:1;           % frequency sweep for plotting [Hz]
w_sweep = 2*pi*freq_axis;
H_sweep = (1i*w_sweep*R*C) ./ (1 + 1i*w_sweep*R*C);

subplot(2,1,1);
plot(freq_axis, abs(H_sweep), 'LineWidth', 1.4); grid on; hold on;
plot(f0, abs(H_k(k==1)), 'ro', 'MarkerFaceColor', 'r');
xlabel('Frequency (Hz)'); ylabel('|H(f)|');
title('RC High-Pass Filter - Magnitude Response');
legend('|H(f)|', 'Signal frequency f_0 = 0.75 Hz', 'Location', 'best');

subplot(2,1,2);
plot(freq_axis, angle(H_sweep)*180/pi, 'LineWidth', 1.4); grid on;
xlabel('Frequency (Hz)'); ylabel('Phase (deg)');
title('RC High-Pass Filter - Phase Response');
```

```
% ------------------------------------------------------------------------
% 3d. Recovered signal after filtering
%     Scale each Fourier component of y(t) by the filter's response at
%     that harmonic frequency, then sum the components back together.
% ------------------------------------------------------------------------
t4 = 0:0.001:2*T0;
y_recovered = zeros(size(t4));
for idx = 1:length(k)
   y_recovered = y_recovered + (c_k(idx)*H_k(idx)) * exp(1i*k(idx)*w0*t4);
end
y_recovered = real(y_recovered);   % discard negligible imaginary round-off

figure('Name','Task 3d: Signal Recovery');
plot(t4, x2_t, 'b', 'LineWidth', 1.6); hold on;
plot(t4, y_orig, 'r:', 'LineWidth', 1.2);
plot(t4, y_recovered, 'k--', 'LineWidth', 1.6); grid on;
xlabel('t (s)'); ylabel('Amplitude (V)');
title('Signal recovery using the RC high-pass filter');
legend('Original x_2(t)', 'Noisy received y(t)', 'Filtered / recovered output', ...
      'Location', 'best');

fprintf('Task 3d: max recovery error vs. original x2(t) = %.4g V\n', ...
       max(abs(y_recovered - x2_t)));
```